

1. Compruebo si existe la base de datos f1, como no existe la creo y me posiciono en ella.

```
hive> show databases;
OK
default
tripdata
Time taken: 0.17 seconds, Fetched: 2 row(s)
hive> CREATE DATABASE f1;
OK
Time taken: 1.164 seconds
hive> show databases;
OK
default
f1
tripdata
Time taken: 0.026 seconds, Fetched: 3 row(s)
hive> use f1;
OK
Time taken: 0.041 seconds
hive>
```

Primero creo la LOCATION para HIVE:

```
hadoop@ab87a6c54b92:~$ hdfs dfs -mkdir -p /user/hive/external/f1/driver_results
hadoop@ab87a6c54b92:~$ hdfs dfs -ls /user/hive/external/f1/driver_results
hadoop@ab87a6c54b92:~$ hdfs dfs -mkdir -p /user/hive/external/f1/constructor_results
```

Creo las carpetas:

```
hive> CREATE EXTERNAL TABLE IF NOT EXISTS f1.driver_results (
  > driver_forename STRING,
  > driver_surname STRING,
  > driver_nationality STRING,
  > points DOUBLE
  > )
  > row format delimited
  > fields terminated by ','
  > stored as textfile
  > location '/user/hive/external/f1/driver_results';
OK
Time taken: 0.435 seconds
```

```
hive> CREATE EXTERNAL TABLE IF NOT EXISTS f1.constructor_results (  
  > constructorRef STRING,  
  > cons_name STRING,  
  > cons_nationality STRING,  
  > url STRING,  
  > points DOUBLE  
  > )  
  > row format delimited  
  > fields terminated by ','  
  > stored as textfile  
  > location '/user/hive/external/f1/constructor_results';  
OK  
Time taken: 0.063 seconds
```

2.

```
hive> DESCRIBE f1.driver_results;  
OK  
driver_forename      string  
driver_surname       string  
driver_nationality   string  
points               double  
Time taken: 0.136 seconds, Fetched: 4 row(s)  
hive> DESCRIBE f1.constructor_results;  
OK  
constructorref       string  
cons_name            string  
cons_nationality     string  
url                  string  
points               double  
Time taken: 0.044 seconds, Fetched: 5 row(s)
```

3.

```
#!/bin/bash
export HADOOP_HOME=/home/hadoop/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
# =====
# 1. VARIABLES DE CONFIGURACION
# =====

# URL de la fuente de los archivos Parquet
BASE_URL="https://data-engineer-edvai-public.s3.amazonaws.com"

# Lista de archivos a descargar
FILES=(
    "results.csv"
    "drivers.csv"
    "constructors.csv"
    "races.csv"
)

# Descarga temporal
DESTINO_TEMPORAL = "/tmp/ingesta_f1"

# Directorio de destino en HDFS
DESTINO_HDFS = "/home/hadoop/landing/f1_data"

# =====
# 2. CONFIGURANDO ENTORNO
# =====

echo "Limpiando y preparando directorio local: ${DESTINO_TEMPORAL}"
rm -rf "${DESTINO_TEMPORAL}"
mkdir -p "${DESTINO_TEMPORAL}"
echo "Directorio local listo."

echo "Verificando/Creando directorio de destino en HDFS: ${DESTINO_HDFS}"
hdfs dfs -mkdir -p "${DESTINO_HDFS}"
echo "Directorio HDFS (Landing Zone) listo."

# =====
# 3. PROCESO DE INGESTA
# =====

for FILE_NAME in "${FILES[@]}; do
    FILE_URL="${BASE_URL}/${FILE_NAME}"
    LOCAL_PATH="${DESTINO_TEMPORAL}/${FILE_NAME}"

    echo "--- Procesando archivo: ${FILE_NAME} ---"

    echo "1. Descargando desde: ${FILE_URL}..."
    wget --no-check-certificate -q --timeout=120 "${FILE_URL}" -O "${LOCAL_PATH}"

    if [ $? -ne 0 ]; then
        echo "ERROR: Fallo la descarga de ${FILE_NAME}. Saliendo."
        rm -f "${LOCAL_PATH}"
        exit 1
    fi

    echo "   Descarga completada en ${LOCAL_PATH}."

    echo "2. Ingestando ${FILE_NAME} a HDFS en ${DESTINO_HDFS}..."
    hdfs dfs -put -f "${LOCAL_PATH}" "${DESTINO_HDFS}"

    echo "   Ingesta completada."

    echo "3. Limpiando archivo local para liberar espacio..."
    rm "${LOCAL_PATH}"
    echo "   Limpieza local de ${FILE_NAME} terminada."
done
```

```

echo ""
echo "=====
echo "EXITO: Ingesta de todos los archivos F1 completada."
echo "Archivos disponibles en HDFS en: ${DESTINO_HDFS}"
echo "=====

# =====
# 4. VERIFICACION FINAL
# =====

echo "--- Verificando archivos finales en HDFS ---"
hdfs dfs -ls "${DESTINO_HDFS}"

```

4.

```

#!/bin/bash
export HADOOP_HOME=/home/hadoop/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
# =====
# 1. VARIABLES DE CONFIGURACION
# =====

# URL de la fuente de los archivos Parquet
BASE_URL="https://data-engineer-edvai-public.s3.amazonaws.com"

# Lista de archivos a descargar
FILES=(
    "results.csv"
    "drivers.csv"
    "constructors.csv"
    "races.csv"
)

# Descarga temporal
DESTINO_TEMPORAL="/tmp/ingesta_f1"

# Directorio de destino en HDFS
DESTINO_HDFS="/home/hadoop/landing/f1_data"

# =====
# 2. CONFIGURANDO ENTORNO
# =====

echo "Limpiando y preparando directorio local: ${DESTINO_TEMPORAL}"
rm -rf "${DESTINO_TEMPORAL}"
mkdir -p "${DESTINO_TEMPORAL}"
echo "Directorio local listo."

```

```

echo "Limpiando directorio de destino en HDFS (Eliminando ${DESTINO_HDFS})..."
hdfs dfs -rm -r -skipTrash "${DESTINO_HDFS}"
echo "Verificando/Creando directorio de destino en HDFS: ${DESTINO_HDFS}"
hdfs dfs -mkdir -p "${DESTINO_HDFS}"
echo "Directorio HDFS (Landing Zone) listo."

# =====
# 3. PROCESO DE INGESTA
# =====

for FILE_NAME in "${FILES[@]"; do
    FILE_URL="${BASE_URL}/${FILE_NAME}"
    LOCAL_PATH="${DESTINO_TEMPORAL}/${FILE_NAME}"

    echo "--- Procesando archivo: ${FILE_NAME} ---"

    echo "1. Descargando desde: ${FILE_URL}..."
    wget --no-check-certificate -q --timeout=120 "${FILE_URL}" -O "${LOCAL_PATH}"

    if [ $? -ne 0 ]; then
        echo "ERROR: Fallo la descarga de ${FILE_NAME}. Saliendo."
        rm -f "${LOCAL_PATH}"
        exit 1
    fi
    echo "   Descarga completada en ${LOCAL_PATH}."

    echo "2. Ingestando ${FILE_NAME} a HDFS en ${DESTINO_HDFS}..."
    hdfs dfs -put -f "${LOCAL_PATH}" "${DESTINO_HDFS}"

    echo "   Ingesta completada."

```

```

    echo "3. Limpiando archivo local para liberar espacio..."
    rm "${LOCAL_PATH}"
    echo "   Limpieza local de ${FILE_NAME} terminada."
done

echo ""
echo "=====
echo "EXITO: Ingesta de todos los archivos F1 completada."
echo "Archivos disponibles en HDFS en: ${DESTINO_HDFS}"
echo "=====

# =====
# 4. VERIFICACION FINAL
# =====

echo "--- Verificando archivos finales en HDFS ---"
hdfs dfs -ls "${DESTINO_HDFS}"

```

5.

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta
from airflow.operators.empty import EmptyOperator

HDFS_PATH = "ssh hadoop@172.17.0.2/home/hadoop/landing/fl_data"
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2025, 1, 1),
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    dag_id='Clase8_JuanI',
    default_args=default_args,
    description='Flujo de Ingesta (Bash) y ETL con Bash (Transformacion y SOBRESCRITURA en Hive).',
    schedule_interval=None,
    catchup=False,
    tags=['ingest', 'transform'],
) as dag:
    start_pipeline = EmptyOperator(
        task_id='start_pipeline',
    )

    ingesta_datos_f1 = BashOperator(
        task_id='ingesta_datos_f1',
        bash_command="bash -c 'bash /home/hadoop/scripts/ingest_f1.sh'",
    )

    spark_etl_process = BashOperator(
        task_id='spark_etl_process',
        bash_command=f"""
            ssh hadoop@172.17.0.2 /home/hadoop/spark/bin/spark-submit \
            --files /home/hadoop/hive/conf/hive-site.xml \
            /home/hadoop/scripts/fl_results.py
        """,
    )

    end_pipeline = EmptyOperator(
        task_id='end_pipeline',
    )

    #source_checks = [check_results, check_constructors, check_drivers, check_races]

    start_pipeline >> ingesta_datos_f1 >> spark_etl_process >> end_pipeline
```

